

1 Analisi dei dati raccolti

I dati raccolti non verranno analizzati solo in valore assoluto, ma verranno **normalizzati rispetto alla propria baseline**. Questo approccio permette di rispondere a domande del tipo: "Quale architettura beneficia maggiormente di un aumento della cache?" o "Quale ISA risulta più efficiente a parità di risorse?".

Il confronto finale avverrà dunque su due livelli:

1. **Intra-ISA:** Confronto tra le tre build della stessa architettura per valutarne la scalabilità.
2. **Inter-ISA:** Confronto tra i rapporti di performance delle tre architetture rispetto alle rispettive baseline, per determinare l'efficienza relativa dei set di istruzioni.

1.1 Analisi delle tre configurazioni per l'architettura ARM

L'analisi delle statistiche relative all'esecuzione del workload su gem5/ARM ha rivelato differenze significative nelle prestazioni e nell'efficienza del sistema, dovute sia alla microarchitettura della CPU, che alla gerarchia di memoria.

<i>Metrica</i>	<i>Baseline (Arm O3)</i>	<i>ArmMinorCPU</i>	<i>Small Caches (O3)</i>
Tempo di simulazione (s)	0,001186	0,005446	0,002132
Istruzioni Simulate	7.807.648	7.807.650	7.807.648
Cicli CPU	2.371.349	10.891.211	4.263.001
IPC (Instructions Per Cycle)	3,292	0,717	1,831
CPI (Cycles Per Instruction)	0,304	1,395	0,546
Branch Cond. Incorrect	4637	4715	4636
Branch Mispredicts (Commit)	4548	N/D*	4548
L1 D-Cache Hits	2.975.995	4.024.561	2.737.002
L1 D-Cache Misses	8943	1878	264.751
L1 D-Cache Miss Rate	0,30%	0,05%	8,82%
L1 I-Cache Hits	572.369	1.113.779	589.353
L1 I-Cache Misses	462	413	468
L1 I-Cache Misses Rate	0,08%	0,04%	0,08%
L2 Cache Hits	60	80	254.708
L2 Cache Misses	2096	2134	2096
L2 Cache Misses Rate	97,22%	96,39%	0,82%

Table 1: Dati di analisi per ARM

La differenza più grande risiede nella scelta del core CPU. Infatti, il modello ArmO3 (Out-Of-Order) è circa 4,6 volte più veloce del modello Minor (In-Order). L'IPC passa invece da 3.29 nella baseline (ottimo risultato) a 0.72, nel caso della variazione del core CPU (Il numero di istruzioni per ciclo si riduce di molto).

Il core O3 riesce ad eseguire più di 3 istruzioni per ciclo grazie all'esecuzione eseguita fuori ordine e grazie anche al parallelismo a livello di istruzione (ILP). Il core Minor, fatica a superare l'istruzione singola in un solo ciclo, a causa del fatto che le istruzioni sono eseguite in-order e probabilmente con meno risorse di issue, e quindi risulta un tempo di esecuzione totale molto più lungo per lo stesso carico di lavoro.

1.1.1 Impatto delle Cache ridotte

Utilizzando delle cache ridotte (con CPU ArmO3) si notano diverse cose:

- **Collo di bottiglia della memoria:** La riduzione della cache ha causato l'aumento della percentuale di miss rate della L1 data cache, passando dallo 0,3% all'8,8%;
- **Effetto sulla gerarchia:** L'aumento dei cache miss di L1 ha spostato il carico sulla cache L2. Si può notare infatti che gli accessi riusciti in L2 passano da 60 a 254.708 tra la prima e l'ultima configurazione;
- **Degrado IPC:** Nonostante la CPU sia la stessa, l'IPC è sceso da 3,29 a 1,83. Ciò succede perché la CPU va in stallo molto più frequentemente, in attesa che i dati vengano recuperati dalla L2 (che è più lenta della L1), quasi dimezzando le prestazioni complessive.

1.1.2 Analisi dei Branch (Previsione dei Salti)

Le statistiche sulla predizione dei rami sono molto simili tra le due configurazioni O3:

- Entrambi hanno lo stesso numero di misprediction (4548), ciò conferma che il workload e il percorso di esecuzione sono identici;
- Il modello Minor CPU mostra un numero leggermente superiore di previsioni errate (4715). Ciò può essere dovuto a un predittore di rami meno sofisticato implementato nel modello Minor rispetto a quello del core O3.

Cond. Incorrect indica il numero di volte in cui il predittore ha sbagliato a prevedere la direzione di un salto condizionale (quelli derivanti da un if o un ciclo for/while), mentre **Branch Mispredicts (Commit)** è un dato più "globale". Include tutti i tipi di salti, non solo quelli condizionali, ma anche i ritorni da funzioni o salti indiretti, che sono arrivati alla fine della pipeline.

Anche se il numero di errori di predizione rimane simile, il costo di ogni errore aumenta drasticamente quando le cache sono piccole.

Nella CPU O3 un salto viene "risolto" solo quando l'istruzione di salto viene eseguita. Se la condizione di salto dipende da un dato che deve essere caricato dalla memoria:

- **Baseline:** il dato può essere trovato in L1 e viene risolto velocemente, altrimenti, se è sbagliato, la CPU se ne accorge e pulisce la pipeline;
- **Small Cache:** Potrebbe esserci un miss in L1. La CPU deve aspettare i cicli della L2, ma nel frattempo continua a fare l'elaborazione lungo il percorso sbagliato, caricando istruzioni inutili.

1.1.3 Branch Mispredicts (Commit)

In questa riga della tabella, per la CPU *ArmMinorCPU*, troviamo la dicitura N/D*.

Questo dato è causato dalla differenza dell'architettura, infatti, nelle CPU O3 esiste uno stadio di Commit (ritiro delle istruzioni).

Il parametro "*system.cpu.commit.branchMispredicts*" conta solo i salti sbagliati che arrivano effettivamente alla fine della pipeline. La Minor CPU è un modello In-Order. Non ha una struttura di Commit separata come il modello O3, quindi la statistica con quel nome non viene generata. Il valore 4715 che si ha nella riga sopra appartiene al parametro "*system.cpu.branchPred.condIncorrect*" (o "*system.cpu.branchPred.mispredict_0::total*").

Sebbene rappresenti tecnicamente le predizioni errate, viene calcolato in una fase diversa

della pipeline (spesso in Execute o Decode) e non è metodologicamente identico al valore di Commit dell'O3.

Il concetto di **Commit**: in un processore moderno, l'esecuzione di un'istruzione avviene in diverse fasi. La fase di Commit (o Retire) è l'ultima, rappresenta **il momento in cui il processore è sicuro al 100% che quell'istruzione debba essere eseguita** scrivendone i risultati permanenti nei registri.

- **Nella CPU O3**: Il processore esegue le istruzioni non nell'ordine in cui appaiono nel codice, ma non appena le risorse sono pronte.

Usa la speculazione: quando incontra un branch (un "if"), scommette sulla direzione (preso o non preso) e continua a lavorare. Se non "vince la scommessa", il processore deve buttare via tutto il lavoro fatto dopo il salto. La statistica "commit.branchMispredicts" conta solo i salti sbagliati che arrivano alla fine della "catena di montaggio". È il dato più preciso perché conta solo gli errori che hanno effettivamente rallentato il programma reale.

- **Nella CPU Minor**: Non esiste una fase di Commit separata e complessa come nell'O3. Le istruzioni fluiscono in modo lineare (Fetch→Decode→Execute), e, non appena il processore scopre che un salto è sbagliato (nello stadio di Execute), ferma tutto e ricomincia. Per questo gem5 non usa il termine "commit" per la Minor: l'istruzione viene "corretta" non appena l'errore è scoperto.

Quindi, **O3** utilizza "*commit.branchMispredicts*" perché è il parametro che misura l'impatto reale sulle prestazioni finali (le istruzioni che hanno completato tutto il ciclo), mentre **Minor** utilizza "*condIncorrect*" perché, pur essendo un modello semplificato "passo-dopo-passo", ogni errore rilevato durante l'esecuzione comporta un reset immediato della pipeline, rendendo superflua la distinzione del "commit".

1.1.4 Cache hit e Cache miss

Nella versione con cache ridotte, i miss della L1 Data Cache aumentano di circa 30 volte.

Il carico si sposta sulla L2 che vede infatti un incremento massiccio degli hit.

La L2 riesce a "tamponare" gran parte dei miss della L1, evitando che la CPU debba andare direttamente in RAM, ma la latenza aggiuntiva della L2 è comunque sufficiente a rallentare l'IPC.

Per quanto riguarda le differenze tra le CPU, la Minor CPU registra un numero di hit (sia I-cache che D-cache) molto più alto rispetto alla baseline O3, nonostante il numero di istruzioni totali sia quasi identico. Questo accade perché il modello Minor (in-order) può generare accessi alla memoria in modo differente o ripetuto, oppure perché la gestione delle istruzioni nel front-end della Minor CPU richiede più fetch per completare lo stesso blocco di codice rispetto alla struttura out-of-order.

I miss della I-Cache rimangono estremamente bassi e costanti in tutte le configurazioni. Questo suggerisce che il codice del workload è molto piccolo e sta comodamente anche in una cache ridotta, mentre sono i dati (D-Cache) a soffrire la riduzione dello spazio.

Curiosamente, il numero di L2 Misses è quasi identico in tutti e tre i casi (circa 2100).

Ciò significa che il set di dati totale necessario al programma entra perfettamente nella L2 (sia quella standard che quella della versione ridotta), e la memoria principale (DRAM) viene

interpellata solo nella fase iniziale di caricamento.

Il fatto che le percentuali siano alte nei primi due casi e molto bassa nel terzo dipende dalla “necessità” di accedere alla cache L2, ovvero: nei primi due casi il rate di miss è molto alto perché non c’è bisogno di utilizzare la cache L2, mentre nel terzo caso si arriva ad utilizzarla per colpa della dimensione ridotta delle cache L1, ma il numero di miss è molto basso.

1.1.5 Conclusioni

L’architettura O3, per questo specifico workload, risulta dominante.

Il parallelismo del core O3 compensa infatti la sua complessità, terminando il compito in una frazione del tempo del core Minor. La dimensione della cache L1 è critica: il workload sembra avere un working set che entra comodamente nella cache L1 standard della baseline, ma non in quella ridotta. La dipendenza dalla L2, nel caso della cache ridotta, introduce una latenza che la CPU O3 non riesce a nascondere completamente portando a un rallentamento del 45% rispetto alla baseline.

Il core Minor è limitato dal front-end, ovvero dalla sua natura “in-order”, non riuscendo a sfruttare le risorse di esecuzione anche quando i dati sono disponibili (considerando anche il basso miss rate delle cache).

1.2 Analisi delle tre configurazioni per l’architettura RISC-V

<i>Metrica</i>	<i>Baseline (RISCV O3)</i>	<i>Minor_RISCV</i>	<i>Small Caches (O3)</i>
Tempo di simulazione (s)	0,003567	0,005997	0,006364
Istruzioni Simulate	10.043.070	10.043.070	10.043.070
Cicli CPU	7.134.755	11.994.951	12.728.034
IPC (Instructions Per Cycle)	1,408	0,837	0,789
CPI (Cycles Per Instruction)	0,710	1,194	1,267
Branch Cond. Incorrect	4816	4898	4817
Branch Mispredicts (Commit)	4725	N/D*	4729
L1 D-Cache Hits	3.551.546	4.050.839	3.315.652
L1 D-Cache Misses	16.645	3681	252.600
L1 D-Cache Miss Rate	0,47%	0,09%	7,08%
L1 I-Cache Hits	600.802	1.150.882	600.828
L1 I-Cache Misses	619	537	629
L1 I-Cache Misses Rate	0,10%	0,05%	0,10%
L2 Cache Hits	40	40	235.932
L2 Cache Misses	3010	3056	3010
L2 Cache Misses Rate	98,69%	98,71%	1,26%

Table 2: Dati di analisi per RISC-V

In RISCV la **CPU O3** è circa 1,7 volte più veloce della **Minor CPU**, parlando di tempo di simulazione. L’IPC della baseline risulta essere 1,408, quindi superiore a poco più di un’istruzione per ciclo, grazie alla capacità di eseguire le istruzioni fuori ordine e parallelizzare le operazioni. Il core Minor, che invece segue l’ordine del codice, subisce più stalli strutturali e dipendenze tra dati, portando il tempo di esecuzione totale a quasi 6ms contro i 3,5ms della baseline.

1.2.1 Impatto delle cache ridotte

La configurazione O3 con le cache ridotte diventa la più lenta, persino della CPU Minor. Il tempo di simulazione è infatti il più alto tra le tre configurazioni, rendendo questa configurazione la meno efficiente. Questo succede perché un core Out-Of-Order è estremamente sensibile alla latenza di memoria.

Quando la L1 D-Cache diventa un collo di bottiglia (il miss rate sale da 0,47% nella baseline a 7,08% nella configurazione con cache ridotte), le strutture interne della CPU si riempiono di istruzioni in attesa dei dati. Inoltre, l'IPC arriva a 0,789.

In questa situazione, la complessità dell'architettura O3 diventa un peso: la CPU occupa più tempo a gestire stalli e speculazioni fallite che a eseguire codice utile.

L'analisi dei flussi di memoria rivela un comportamento gerarchico ben definito.

La I-Cache rimane più o meno stabile in tutte le configurazioni, ciò indica che il codice eseguibile è piccolo e risiede agevolmente anche in cache ridotte.

Per quanto riguarda la L1 D-Cache, nella baseline essa filtra quasi tutto. Alla L2 arrivano circa 3000 richieste, che però falliscono quasi tutte (infatti il miss rate è al 98,7%), ma si tratta dei **Compulsory Misses**, ovvero dati nuovi mai visti prima.

Nella versione Small Caches, la L1 espelle molti dati. La L2 viene investita da oltre 235.000 accessi (Hits) che erano gestiti dalla L1 nelle altre configurazioni. Il Miss Rate della L2 scende all'1,26%, ma la L2 sta solo facendo il lavoro che la L1 non può più fare, introducendo una latenza maggiore che rallenta l'intero sistema.

1.2.2 Analisi dei Branch (Previsione dei Salti)

Il numero di predizioni errate è molto simile tra le versioni O3, segno che il percorso logico del software non cambia. Nella configurazione Small caches, pur avendo lo stesso numero di errori, la CPU paga un prezzo maggiore in termini di cicli di stallo perché impiega più tempo recuperare i dati necessari a risolvere la condizione del salto (a causa dei miss in L1);

1.2.3 Conclusioni

In architetture RISC-V, un core potente (O3) senza una cache adeguata è controproducente. In scenari con memoria limitata, un core più semplice (In-Order) può risultare più veloce e prevedibile. Il limite del sistema è chiaramente la L1 Data Cache.

Le dimensioni della L1 Instruction Cache e della L2 sono invece adeguate per questo specifico carico di lavoro. I circa 3000 miss in L2 rappresentano il limite fisico del workload, ovvero la quantità minima di dati che deve essere caricata dalla memoria principale (DRAM) indipendentemente dall'efficienza dei core.

1.3 Analisi delle tre configurazioni per l'architettura MarssX86

1.4 Analisi delle baseline tra le architetture